

Program - 1

1: Implement and Demonstrate Depth First Search Algorithm on Water Jug Problem

Problem Statement

: Water Jug problem in Artificial Intelligence

- ✓ In the Water jug problem in Artificial Intelligence, we are provided with two jug:
- ✓ One is having the capacity to hold 4 liters of water and the other has the capacity to hold 3 liters
- ✓ There is no other measuring equipment available and the jug also do not have any kind of marking on them
- ✓ So the agent's task is to fill the 4 liters' jug with 2 liters of water any these two jugs and no other material
- ✓ Initially, both our jugs are empty so to solve this problem following set of rules were proposed

Production rules for solving the water jug problem

Here let x denote the 4- liter jug and y denote the 3- liter-jug

Program code

```
#initialize the jug and thier capacity
a = int(input("Enter Jug A Capacity: "))
b = int(input("Enter Jug B Capacity: "))
ai = int(input("Initially Water in Jug A: "))
bi = int(input("Initially Water in Jug B: "))
af = int(input("Final State of Jug A: "))
bf = int(input("Final State of Jug B: "))
#print List Of Operations
print("List Of Operations You can Do:\n")
print("1.Fill Jug A Completely\n")
```

```
print("2.Fill Jug B Completely\n")
print("3.Empty Jug A Completely\n")
print("4.Empty Jug B Completely\n")
print("5.Pour From Jug A till Jug B filled Completely or A becomes empty\n")
print("6.Pour From Jug B till Jug A filled Completely or B becomes empty\n")
print("7.Pour all From Jug B to Jug A\n")
print("8.Pour all From Jug A to Jug B\n")
#loop
while ((ai != af or bi != bf)):
    op = int(input("Enter the Operation: "))
    if (op == 1):
        ai = a
    elif (op == 2):
        bi = b
    elif (op == 3):
        ai = 0
    elif (op == 4):
        bi = 0
    elif (op == 5):
        if (b-bi > ai):
            bi = ai+bi
            ai = 0
        else:
            ai = ai-(b-bi)
            bi = b
    elif (op == 6):
        if (a-ai > bi):
            ai = ai+bi
            bi = 0
        else:
```

$b_i = b_i - (a - a_i)$

$a_i = a$

elif (op == 7):

$a_i = a_i + b_i$

$b_i = 0$

elif (op == 8):

$b_i = b$

output

Enter Jug A Capacity: 4

Enter Jug B Capacity: 3

Initially Water in Jug A: 0

Initially Water in Jug B: 0

Final State of Jug A: 0

Final State of Jug B: 2

List Of Operations You can Do:

1.Fill Jug A Completely

2.Fill Jug B Completely

3.Empty Jug A Completely

4.Empty Jug B Completely

5.Pour from Jug A Till Jug B filled Completely or A becomes empty

6.Pour from Jug B till Jug A Filled Completely or B becomes empty

7.Pour all from Jug B to Jug A

8.Pour all from Jug A to Jug B

Enter the Operation: 2

0 3

Enter the Operation: 6

3 0

Enter the Operation: 2

3 3

Enter the Operation: 6

4 2

Enter the Operation: 3

0 2

Process finished with exit code 0

Program - 2

2: Implement and Demonstrate Best First Algorithm on Missionaries- cannibals Problems using python

Problem Statement

- ✓ Three missionaries and three cannibals are on one side of a river, along with about that can hold one or two people
- ✓ Find a way to get everyone to the other side without ever leaving a group of missionaries in one place outnumbered by the cannibals in the place
- ✓ The boat cannot cross the river by itself with no people on board

Program code

```
print("\n")

print("\tGame Start\nNow the task is to move all of them to right side of the river")

print("rules:\n1. The boat can carry at most two people\n2. If cannibals num greater than missionaries then the cannibals would eat the missionaries\n3. The boat cannot cross the river by itself with no people on board")

lM = 3
lC = 3
rM = 0
rC = 0

userM = 0 # userM = User input for number of missionaries for right to left side travel
userC = 0 # userC = User input for number of cannibals for right to left travel
k = 0

print("\nM M M C C C |   --- | \n")

try:
    while (True):
        while (True):
            print("Left side -> right side river travel")
            # uM = user input for number of missionaries for left to right travel
            # uC = user input for number of cannibals for left to right travel
```

```

uM = int(input("Enter number of Missionaries travel => "))
uC = int(input("Enter number of Cannibals travel => "))

if ((uM == 0) and (uC == 0)):
    print("Empty travel not possible")
    print("Re-enter : ")
elif (((uM + uC) <= 2) and ((lM - uM) >= 0) and ((lC - uC) >= 0)):
    break
else:
    print("Wrong input re-enter : ")

lM = (lM - uM)
lC = (lC - uC)
rM += uM
rC += uC

print("\n")
for i in range(0, lM):
    print("M ", end="")
for i in range(0, lC):
    print("C ", end="")
print("| --> | ", end="")
for i in range(0, rM):
    print("M ", end="")
for i in range(0, rC):
    print("C ", end="")
print("\n")

k += 1

if (((lC == 3) and (lM == 1)) or ((lC == 3) and (lM == 2)) or ((lC == 2) and (lM == 1))
or (

```

```
        (rC == 3) and (rM == 1)) or ((rC == 3) and (rM == 2)) or ((rC == 2) and (rM == 1)))
```

```
print("Cannibals eat missionaries:\nYou lost the game")
```

```
break
```

```
if ((rM + rC) == 6):
```

```
    print("You won the game : \n\tCongrats")
```

```
    print("Total attempt")
```

```
    print(k)
```

```
    break
```

```
while (True):
```

```
    print("Right side -> Left side river travel")
```

```
    userM = int(input("Enter number of Missionaries travel => "))
```

```
    userC = int(input("Enter number of Cannibals travel => "))
```

```
    if ((userM == 0) and (userC == 0)):
```

```
        print("Empty travel not possible")
```

```
        print("Re-enter : ")
```

```
    elif (((userM + userC) <= 2) and ((rM - userM) >= 0) and ((rC - userC) >= 0)):
```

```
        break
```

```
    else:
```

```
        print("Wrong input re-enter : ")
```

```
    lM += userM
```

```
    lC += userC
```

```
    rM -= userM
```

```
    rC -= userC
```

```
    k += 1
```

```
    print("\n")
```

```
    for i in range(0, lM):
```

```

        print("M ", end="")
    for i in range(0, lC):
        print("C ", end="")
    print("| <-- | ", end="")
    for i in range(0, rM):
        print("M ", end="")
    for i in range(0, rC):
        print("C ", end="")
    print("\n")

    if (((lC == 3) and (lM == 1)) or ((lC == 3) and (lM == 2)) or ((lC == 2) and (lM == 1))
or (
        (rC == 3) and (rM == 1)) or ((rC == 3) and (rM == 2)) or ((rC == 2) and (rM ==
1)))):
        print("Cannibals eat missionaries:\nYou lost the game")
        break
except EOFError as e:
    print("\nInvalid input please retry !!")

```


output

Game Start

Now the task is to move all of them to right side of the river

rules:

1. The boat can carry at most two people
2. If cannibals numb greater than missionaries then the cannibals would eat the missionaries
3. The boat cannot cross the river by itself with no people on board

M M M C C C | --- |

Left side -> right side river travel

Enter number of Missionaries travel => 0

Enter number of Cannibals travel => 2

M M M C | --> | C C

Right side -> Left side river travel

Enter number of Missionaries travel => 0

Enter number of Cannibals travel => 1

M M M C C | <-- | C

Left side -> right side river travel

Enter number of Missionaries travel => 0

Enter number of Cannibals travel => 2

M M M | --> | C C C

Right side -> Left side river travel

Enter number of Missionaries travel => 0

Enter number of Cannibals travel => 1

M M M C | <-- | C C

Left side -> right side river travel

Enter number of Missionaries travel => 2

Enter number of Cannibals travel => 0

M C | --> | M M C C

Right side -> Left side river travel

Enter number of Missionaries travel => 1

Enter number of Cannibals travel => 1

M M C C | <-- | M C

Left side -> right side river travel

Enter number of Missionaries travel => 2

Enter number of Cannibals travel => 0

C C | --> | M M M C

Right side -> Left side river travel

Enter number of Missionaries travel => 0

Enter number of Cannibals travel => 1

C C C | <-- | M M M

Left side -> right side river travel

Enter number of Missionaries travel => 0

Enter number of Cannibals travel => 2

C | --> | M M M C C

Right side -> Left side river travel

Enter number of Missionaries travel => 1

Enter number of Cannibals travel => 0

M C | <-- | M M C C

Left side -> right side river travel

Enter number of Missionaries travel => 1

Enter number of Cannibals travel => 1

| --> | M M M C C C

You won the game :

Congrats

Total attempt

11

Process finished with exit code 0

Program – 3

3. Implement A* Search Algorithm

Problem Statement

- ✓ The problem statement for the A* search algorithm is to find the optimal path (shortest distance or lowest cost),
- ✓ The Statement from a given start node to a specified goal node within a graph, by utilizing a heuristic function that estimates the remaining cost to reach the goal from any given node, thus guiding the search towards the most promising paths

Program code

```
class Node:
    def __init__(self, name, parent=None):
        self.name = name
        self.parent = parent
        self.g = 0 # Cost from start to this node
        self.h = 0 # Heuristic cost from this node to goal
        self.f = 0 # Total cost

def a_star(start, goal, graph):
    open_set = [start]
    closed_set = []

    while open_set:
        current_node = min(open_set, key=lambda node: node.f)

        if current_node.name == goal.name:
```

```
path = []
while current_node:
    path.append(current_node.name)
    current_node = current_node.parent
return path[::-1] # Return reversed path
```

```
open_set.remove(current_node)
closed_set.append(current_node)
```

```
for neighbor in graph[current_node.name]:
    if neighbor in closed_set:
        continue
```

```
tentative_g = current_node.g + 1 # Assuming cost between nodes is 1
```

```
if neighbor not in open_set:
    open_set.append(neighbor)
elif tentative_g >= neighbor.g:
    continue
```

```
neighbor.parent = current_node
neighbor.g = tentative_g
neighbor.h = heuristic(neighbor, goal)
neighbor.f = neighbor.g + neighbor.h
```

```
return None # No path found
```

```
def heuristic(node, goal):
```

```
    # Simple heuristic: straight-line distance (can be improved)
```

```
    return abs(ord(node.name) - ord(goal.name))
```

Example usage

```
graph = {  
    'A': [Node('B'), Node('C')],  
    'B': [Node('A'), Node('D')],  
    'C': [Node('A'), Node('D')],  
    'D': [Node('B'), Node('C'), Node('E')],  
    'E': [Node('D')]  
}
```

```
start_node = Node('A')  
goal_node = Node('E')  
path = a_star(start_node, goal_node, graph)  
print("Path found:", path)
```

output

Path found: ['A', 'C', 'D', 'E']

Process finished with exit code 0

Program – 4

4. Implement AO * Search Algorithm

Problem Statement

- ✓ AO* algorithm is a best first search algorithm. AO* algorithm uses the concept of AND-OR graphs to decompose any complex problem given into smaller set of problems which are further solved
- ✓ AND-OR graphs are specialized graphs that are used in problems that can be broken down into sub problems where
- ✓ AND side of the graph represent a set of task that need to be done to achieve the main goal
- ✓ whereas the or side of the graph represent the different ways of performing task to achieve the same main goal.

Program code

```
# Cost to find the AND and OR path
def Cost(H, graph, weight=1):
    cost = {}
    if 'AND' in graph:
        AND_nodes = graph['AND']
        Path_A = ' AND '.join(AND_nodes)
        PathA = sum(H[node] + weight for node in AND_nodes)
        cost[Path_A] = PathA

    if 'OR' in graph:
        OR_nodes = graph['OR']
        Path_B = ' OR '.join(OR_nodes)
```

```

    PathB = min(H[node] + weight for node in OR_nodes)
    cost[Path_B] = PathB
return cost

# Update the cost
def update_cost(H, graph, weight=1):
    Main_nodes = list(graph.keys())
    Main_nodes.reverse()
    least_cost = {}
    for key in Main_nodes:
        condition = Conditions[key]
        print(key, ':', Conditions[key], '>>>', Cost(H, condition, weight))
        c = Cost(H, condition, weight)
        H[key] = min(c.values())
        least_cost[key] = Cost(H, condition, weight)
    return least_cost

# Print the shortest path
def shortest_path(Start, Updated_cost, H):
    Path = Start
    if Start in Updated_cost.keys():
        Min_cost = min(Updated_cost[Start].values())
        key = list(Updated_cost[Start].keys())
        values = list(Updated_cost[Start].values())
        Index = values.index(Min_cost)

        # FIND MINIMUM PATH KEY
        Next = key[Index].split()

        # ADD TO PATH FOR OR PATH

```

```

    if len(Next) == 1:

        Start = Next[0]

        Path += '<--' + shortest_path(Start, Updated_cost, H)
    # ADD TO PATH FOR AND PATH
    else:

        Path += '<--(' + key[Index] + ') '

        Start = Next[0]

        Path += '[' + shortest_path(Start, Updated_cost, H) + ' + '

        Start = Next[-1]

        Path += shortest_path(Start, Updated_cost, H) + ']'

    return Path

H = {'A': -1, 'B': 5, 'C': 2, 'D': 4, 'E': 7, 'F': 9, 'G': 3, 'H': 0, 'I': 0, 'J': 0}
Conditions = {
    'A': {'OR': ['B'], 'AND': ['C', 'D']},
    'B': {'OR': ['E', 'F']},
    'C': {'OR': ['G'], 'AND': ['H', 'I']},
    'D': {'OR': ['J']}
}

# weight
weight = 1

# Updated cost
print("Updated Cost :")

Updated_cost = update_cost(H, Conditions, weight=1)

print('-----')

print('Shortest Path :\n', shortest_path('A', Updated_cost, H))

```


output

```
Updated Cost :  
D : {'OR': ['J']} >>> {'J': 1}  
C : {'OR': ['G'], 'AND': ['H', 'I']} >>> {'H AND I': 2, 'G': 4}  
B : {'OR': ['E', 'F']} >>> {'E OR F': 8}  
A : {'OR': ['B'], 'AND': ['C', 'D']} >>> {'C AND D': 5, 'B': 9}  
-----  
Shortest Path :  
A<--(C AND D) [C<--(H AND I) [H + I] + D<--J]
```

Program – 5

5 . Solve 8- Queens Problem with suitable assumptions

Problem statement

- ✓ The 8-Queens problem can be solved by placing 8 chess queens on an 8x8 chessboard so that no two queens attacked each other (i.e., no two queens are in the same row, column, or diagonal).
- ✓ A common approach is to use a backtracking algorithm, which recursively tries placing queens on the board, and backtracks if a placement leads to a conflict

Program code

```
def solve_n_queens(n):
    def is_safe(board, row, col):
        for r in range(row):
            if board[r] == col or abs(board[r] - col) == abs(r - row):
                return False
        return True

    def place_queen(board, row):
        if row == n:
            solutions.append(board[:])
            return
        for col in range(n):
            if is_safe(board, row, col):
                board[row] = col
                place_queen(board, row + 1)
                board[row] = -1

    board = [-1] * n
    solutions = []
    place_queen(board, 0)
    return solutions
```

```
def print_board(board):
```

```

n = len(board)
for row in range(n):
    line = ""
    for col in range(n):
        if board[row] == col:
            line += "Q "
        else:
            line += "- "
    print(line)
print()

# Find all solutions for 8 queens
n = 8
all_solutions = solve_n_queens(n)
# Print all solutions
print(f"Total solutions for {n} queens: {len(all_solutions)}")
for idx, solution in enumerate(all_solutions, start=1):
    print(f"Solution {idx}:")
    print_board(solution)

```

Out put

```

Solution 1:
Q-----
----Q---
-----Q
-----Q--
--Q-----
-----Q-
-Q-----
---Q-----
Solution 2:
Q-----
-----Q--
-----Q
--Q-----
-----Q-
---Q-----
-Q-----
----Q---
.
.
.
.

```

```

Solution 91:
-----Q
--Q-----
Q-----

```

```
-----Q--  
-Q-----  
----Q---  
-----Q-  
---Q----
```

Solution 92:

```
-----Q  
---Q----  
Q-----  
--Q-----  
-----Q--  
-Q-----  
-----Q-  
----Q----
```

Process finished with exit code 0

Program – 6

6 : Implementation of TSP using heuristic approach

Problem statement

- ✓ The traveling salesman problem consists of a salesman and a set of cities.
- ✓ The salesman has to visit each one of the cities starting from a certain one (e.g. the hometown) and returning to the same city.
- ✓ The challenge of the problem is that the traveling salesman wants to minimize the total length of the trip.

Program code

```
import numpy as np
import random

n=5

mat =[[0 for i in range(n)] for j in range(n)]

for i in range(n):
    for j in range(n):
        if i!=j:
            x = int(input(f'Enter distance between location {i} and location {j} : '))
            mat[i][j] = x

print('Distances Matrix is :')
print('\n'.join(['\t'.join([str(cell) for cell in row]) for row in mat]))

start= random.randint(0,n-1)
tour=[start]
total_distance=0
#using nearest neighbor algorithm as heuristic
for i in range(n-1):

    m=next(x for x in list(range(0,n)) if x not in tour)

    for j in range(n):
        if j not in tour:
            if mat[start][j] < mat[start][m]:
                m=j

    total_distance+=mat[start][m]
    tour.append(m)
    start = m
```

```
tour.append(tour[0])
total_distance+=mat[start][tour[0]]

print(f"\nStart location : {tour[0]}")
print("Tour :",tour)
print(f"Total distance is : {total_distance}")
```

Out put

Enter distance between location 0 and location 1 : 50
Enter distance between location 0 and location 2 : 60
Enter distance between location 0 and location 3 : 70
Enter distance between location 0 and location 4 : 80
Enter distance between location 1 and location 0 : 40
Enter distance between location 1 and location 2 : 70
Enter distance between location 1 and location 3 : 60
Enter distance between location 1 and location 4 : 50
Enter distance between location 2 and location 0 : 40
Enter distance between location 2 and location 1 : 50
Enter distance between location 2 and location 3 : 60
Enter distance between location 2 and location 4 : 80
Enter distance between location 3 and location 0 : 90
Enter distance between location 3 and location 1 : 50
Enter distance between location 3 and location 2 : 60
Enter distance between location 3 and location 4 : 70
Enter distance between location 4 and location 0 : 80
Enter distance between location 4 and location 1 : 80
Enter distance between location 4 and location 2 : 70
Enter distance between location 4 and location 3 : 80

Distances Matrix is :

0	50	60	70	80
40	0	70	60	50
40	50	0	60	80
90	50	60	0	70
80	80	70	80	0

Start location : 2

Tour : [2, 0, 1, 4, 3, 2]

Total distance is : 280

Process finished with exit code 0

Program – 7

7: Implementation of the problem solving strategies: either using Forward Chaining or Backward Chaining

Problem statement

- ✓ To implement either forward or backward chaining for problem-solving, you'll need to define the problem's knowledge base, rules, and a goal state.
- ✓ Forward chaining starts with known facts and uses rules to infer new facts until a goal is reached, while backward chaining starts with a goal and works backward to find the necessary facts and rules to support it.

Program code

```
def forward_chaining (rules, facts, goal):
    inferred_facts = set(facts)
    new_facts = True
    while new_facts:
        new_facts = False
        for rule in rules:
            condition, result = rule
            if all(cond in inferred_facts for cond in condition) and result not in inferred_facts:
                inferred_facts.add(result)
        new_facts = True
    if result == goal:
        return True
    return False

def backward_chaining (rules, facts, goal):
    def ask(query):
        if query in facts:
            return True
```

```
for rule in rules:
    condition, result = rule
    if result == query and all(ask(cond) for cond in condition):
        return True
    return False
return ask(goal)
```

```
# Define the rules and facts
```

```
rules = [
    (['hair', 'live young'], 'mammal'),
    (['feathers', 'fly'], 'bird')
]
facts = ['hair', 'live young']
goal = 'mammal'
```

```
# Use forward chaining
```

```
is_mammal = forward_chaining(rules, facts, goal)
if is_mammal:
    print("The cat is classified as a mammal.")
else:
    print("The cat is not classified as a mammal.")
facts = ['feathers', 'fly']
goal = 'bird'
```

```
# Use backward chaining
```

```
is_bird = backward_chaining(rules, facts, goal)
if is_bird:
    print("The pigeon is classified as a bird.")
else:
    print("The pigeon is not classified as a bird.")
```


out put

The cat is classified as a mammal.

The pigeon is classified as a bird

Program – 8

8 : Implement resolution principle on FOPL related problems

Problem statement

- ✓ Given a set of FOPL clauses and a target statement (conclusion) to be proved, implement the resolution principle to determine if the conclusion follows logically from the given clauses.

Program code

```
def negate_literal(literal):
    if literal.startswith("~"):
        return literal[1:]
    else:
        return "~" + literal

def resolve(clause1, clause2):
    new_clauses = []
    for literal1 in clause1:
        for literal2 in clause2:
            if literal1 == negate_literal(literal2):
                new_clause = set(clause1 + clause2)
                new_clause.remove(literal1)
                new_clause.remove(literal2)
                new_clauses.append(list(new_clause))
    return new_clauses

def resolution(knowledge_base, goal):
    clauses = [clause.split(" v ") for clause in knowledge_base]
    clauses.append([negate_literal(lit) for lit in goal.split(" v ")])

    while True:
```

```

new_clauses = []
for i in range(len(clauses)):
    for j in range(i + 1, len(clauses)):
        resolved_clauses = resolve(clauses[i], clauses[j])
        if [] in resolved_clauses:
            return True # Contradiction found, goal is proven
        new_clauses.extend(resolved_clauses)

    if all(clause in clauses for clause in new_clauses):
        return False # No new clauses can be generated, goal cannot be proven

for clause in new_clauses:
    if clause not in clauses:
        clauses.append(clause)

knowledge_base = ["P v Q", "~P v R", "~Q v S"]

goal = "~R v S"

if resolution(knowledge_base, goal):

    print(f"Goal '{goal}' is proven.")

else:
    print(f"Goal '{goal}' cannot be proven.")

```

out put

Goal 'R v S' is proven.

Process finished with exit code 0

Program – 9

9: Implement Tic-Tac-Toe game using Python

Problem statement

- ✓ A Tic-Tac-Toe game involves designing a two-player game played on a 3x3 grid.
- ✓ Players take turns placing 'X' or 'O' in empty cells.
- ✓ The first player to get three in a row (horizontally, vertically, or diagonally) wins, or the game ends in a draw if all cells are filled

Program code

```
board = [" " for x in range(9)]
def print_ board():
row1 = "| {} | {} | {} |".format(board[0], board[1], board[2])
row2 = "| {} | {} | {} |".format(board[3], board[4], board[5])
row3 = "| {} | {} | {} |".format(board[6], board[7], board[8])
print(row1)
print(row2)
print(row3)
print()
def player_ move(icon):
if icon == "X":
number = 1
elif icon == "O":
number = 2
print("Your turn player {}".format(number))
choice = int(input("Enter your move (1-9): ").strip())
if board[choice - 1] == " ":
board[choice - 1] = icon
else:
print()
print("That space is already taken!")
def is victory (icon):
if (board[0] == icon and board[1] == icon and board[2] == icon) or \
(board[3] == icon and board[4] == icon and board[5] == icon) or \
```

```

(board[6] == icon and board[7] == icon and board[8] == icon) or \
(board[0] == icon and board[3] == icon and board[6] == icon) or \
(board[1] == icon and board[4] == icon and board[7] == icon) or \
(board[2] == icon and board[5] == icon and board[8] == icon) or \
(board[0] == icon and board[4] == icon and board[8] == icon) or \
(board[2] == icon and board[4] == icon and board[6] == icon):
    return True
else:

    return False
def is_draw():
    if " " not in board:
        return True
    else:

        return False
while True:
    print_board()
    player_move("X")
    print_board()
    if is_victory("X"):
        print("X wins! Congratulations!")
        break

    elif is_draw():
        print("It's a draw!")
        break

    player_move("O")
    if is_victory("O"):
        print_board()
        print("O wins! Congratulations!")
        break

    elif is_draw():
        print("It's a draw!")
        break

```

out put

```
| | | |  
| | | |  
| | | |
```

Your turn player 1

Enter your move (1-9): 1

```
| X | | |  
| | | |  
| | | |
```

Your turn player 2

Enter your move (1-9): 2

```
| X | O | |  
| | | |  
| | | |
```

Your turn player 1

Enter your move (1-9): 4

```
| X | O | |  
| X | | |  
| | | |
```

Your turn player 2

Enter your move (1-9): 7

```
| X | O | |  
| X | | |  
| O | | |
```

Your turn player 1

Enter your move (1-9): 5

```
| X | O | |
```

| X | X | |

| O | | |

Your turn player 2

Enter your move (1-9): 9

| X | O | |

| X | X | |

| O | | O |

Your turn player 1

Enter your move (1-9): 3

| X | O | X |

| X | X | |

| O | | O |

Your turn player 2

Enter your move (1-9): 8

| X | O | X |

| X | X | |

| O | O | O |

O wins! Congratulations!

Process finished with exit code 0

Program – 10

10 : - Build a bot which provides all the information related to text in search box.

Problem statement

Program code

```
import necessary libraries
import io
import random
import string # to process standard python strings
import warnings
import numpy as np
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity
import warnings
warnings.filterwarnings('ignore')
import nltk
from nltk.stem import WordNetLemmatizer
# Reading in the corpus
with open('chatbot.txt', 'r', encoding='utf8', errors='ignore') as fin:
    raw = fin.read().lower()
# TOfenisation
sent_tokens = nltk.sent_tokenize(raw) # converts to list of sentences
word_tokens = nltk.word_tokenize(raw) # converts to list of words
```



```

# Preprocessing
lemmer = WordNetLemmatizer()

def LemTokens(tokens):
    return [lemmer.lemmatize(token) for token in tokens]

remove_punct_dict = dict((ord(punct), None) for punct in string.punctuation)

def LemNormalize(text):
    return LemTokens(nltk.word_tokenize(text.lower().translate(remove_punct_dict)))

# Keyword Matching
GREETING_INPUTS = ("hello", "hi", "greetings", "sup", "what's up", "hey",)
GREETING_RESPONSES = ["hi", "hey", "*nods*", "hi there", "hello", "I am glad! You are talking to me"]

def greeting(sentence):
    """If user's input is a greeting, return a greeting response"""
    for word in sentence.split():
        if word.lower() in GREETING_INPUTS:
            return random.choice(GREETING_RESPONSES)

# Generating response
def response(user_response):
    robo_response = ""
    sent_tokens.append(user_response)
    TfidfVec = TfidfVectorizer(tokenizer=LemNormalize, stop_words='english')
    tfidf = TfidfVec.fit_transform(sent_tokens)
    vals = cosine_similarity(tfidf[-1], tfidf)
    idx = vals.argsort()[0][-2]
    flat = vals.flatten()
    flat.sort()
    req_tfidf = flat[-2]
    if (req_tfidf == 0):
        robo_response = robo_response + "I am sorry! I don't understand you"
    return robo_response
else:

```

```

robo_response = robo_response + sent_tokens[idx]

return robo_response

flag = True

print("ROBOT: My name is Robot. I will answer your queries about Chatbots. If you want to
exit, type Bye!")

while (flag == True):

user_response = input()

user_response = user_response.lower()

if (user_response != 'bye'):

if (user_response == 'thanks' or user_response == 'thank you'):

flag = False

print("ROBOT: You are welcome..")

else:

if (greeting(user_response) != None):

print("ROBOT: " + greeting(user_response))

else:

print("ROBOT: ", end="")

print(response(user_response))

sent_tokens.remove(user_response)

else:

flag = False

print("ROBOT: Bye! take care..")

```

out put

ROBOT: My name is Robot. I will answer your queries about Chatbots. If you want to exit,
type Bye!

hi

ROBOT: hey

dbit

ROBOT:

dbit is an engineering college in bangalore and it is located in mysuru road